

SP – LAB nr 7 - 20.05.2026 - Materiały

Agenda zajęć

Część A: Kickoff projektu i zasady zaliczenia — 15 min

- Cel LAB 07: rozpoczęcie projektu zaliczeniowego „Nadzór Archiwum Dowodowego” (Evidence Room Surveillance Node).
- Deklaracja poziomu realizacji: **BASIC, PERFORMANCE, DELIGHTER**.
- Omówienie protokołu obrony projektu: prywatne repozytorium, folder `/zaliczenie`, raport PDF oraz test na LAB 08.

Część B: Demo techniczne — Wireshark + MQTT Awareness — 20 min

- Pokaz toru: `Sensor Node (C++ Arduino) -> Bridge COM/HTTP (C#) -> Client / Serwis alarmowy C#`.
- Pokaz w Wiresharku: nieszyfrowany HTTP pokazuje payload JSON i nagłówki w postaci jawnej.
- Krótkie wprowadzenie do MQTT: publish-subscribe, broker, low-overhead.
- Granica zakresu: MQTT jest tylko zajawką architektoniczną, nie wymaganiem implementacyjnym.

Część C: Warsztat projektowy i start kodowania — 55 min

- Przygotowanie folderu `/zaliczenie`.
- Uruchomienie Sensor Node w Wokwi / VS Code / PlatformIO.
- Konfiguracja mostu: Serial z węzła do Bridge COM/HTTP.
- Start Klienta / Consumer Service w C#.
- Pierwszy przepływ danych: JSON z dwóch czujników -> Bridge -> konsola klienta.

Artefakt: działający projekt zaliczeniowy `Sensor Node (C++ Arduino) -> Bridge COM/HTTP (C#) -> Client / Serwis alarmowy C#`, w którym dane z BMP180 i PIR są łączone w jeden payload, przechodzą przez Bridge i są analizowane po stronie aplikacji C#.

Opis zadania zaliczeniowego

Temat projektu: Nadzór Archiwum Dowodowego (Evidence Room Surveillance Node).

Context: W projekcie zaliczeniowym składasz wcześniejsze elementy kursu w jeden scenariusz systemowy. Pracowałeś już z danymi pomiarowymi, zdarzeniem z PIR, Bridge'em, HTTP, C#, CSV/logiem i prostą obserwowalnością. Teraz te elementy mają utworzyć mały system nadzoru strefy.

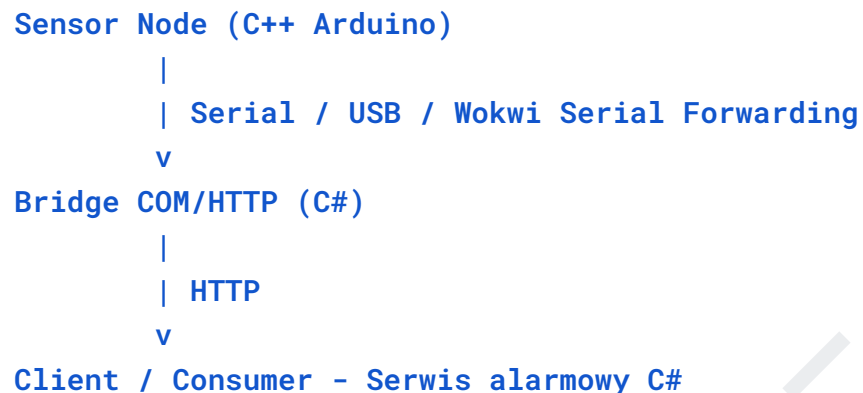
Problem biznesowy: Pomieszczenia z dowodami rzeczowymi muszą być hermetyczne i monitorowane. Naruszenie strefy nie zawsze wygląda jak jeden oczywisty sygnał. Może pojawić się jako ruch w niewłaściwym czasie, gwałtowny spadek ciśnienia, albo kombinacja danych, które dopiero razem mają znaczenie.

W projekcie obserwujesz dwa wektory zagrożeń:

- **rozszerzenie / wybite szyby / wyrwanie elementu wentylacji** — modelowane przez gwałtowny spadek ciśnienia z BMP180,
- **obecność intruza** — modelowana przez aktywację czujnika PIR.

Fuzja danych (data fusion): Sensor Node nie wysyła dwóch luźnych komunikatów. Ma przygotować jeden spójny payload JSON, w którym widać stan środowiska. Client / Serwis alarmowy C# ma umieć ten payload odczytać, zaprezentować i — w wariancie PERFORMANCE — rozpoznać naruszenie strefy (facility breach).

Architektura projektu:



Uwaga techniczna: Wykorzystujemy wzorzec Bridge znany z Labu nr 4: **Serial** -> **Bridge** -> **HTTP**. Powód jest praktyczny: w środowisku laboratoryjnym i w Wokwi Personal nie mamy łatwej realizacji scenariusza Wi-Fi a za tym też wykorzystani serwera HTTP działającego bezpośrednio na wirtualnym mikrokontrolerze. Bridge jest kontrolowanym adapterem między światem urządzenia podłączonego przez Serial a światem aplikacji C# konsumującej dane przez HTTP.

Inżynierskie „Dlaczego?”: W systemie pomiarowym nie wystarczy pokazać liczby w konsoli. Dane muszą przejść przez jawny tor, zachować sens po drodze i zostać użyte do decyzji. Jeżeli system zgłasza naruszenie archiwum dowodowego, musi być wiadomo: z jakich danych wynika alarm, kiedy wystąpił i dlaczego został zapisany.



Zasady zaliczenia

Projekt z LAB 07 stanowi **50% całkowitej puli punktów z laboratorium**. Pozostałe **50%** to test na LAB 08. Przystąpienie do testu możliwe gdy zostało wykonane zadanie zaliczeniowe i udostępnione na GitHub.

1. Protokół Obrony (Project Defense Protocol)

Na Teście pojawią się pytania otwarte bezpośrednio dotyczące projektu zaliczeniowego. Test weryfikuje, m.in. czy rozumiesz własną architekturę, kontrakt JSON, podział odpowiedzialności i decyzje techniczne.

Przykłady pytań (będą inne):

- gdzie powstaje payload JSON,
- co robi Bridge COM/HTTP,

Brak wiedzy o decyzjach we własnym kodzie unieważnia punkty za projekt. To nie jest kara za pomyłkę składniową. To jest konsekwencja oddania projektu, którego nie potrafisz obronić.

2. Repozytorium

Kod musi zostać umieszczony w Twoim prywatnym repozytorium GitHub które zostało stworzone specjalnie na te zajęcia, w nowym folderze: `/zaliczenie`. Raport PDF w `/zaliczenie/raport`. W folderze zaliczenie ma znajdować się plik readme opisujący jak uruchomić projekt (`/zaliczenie/Readme.md`).

Folder `/zaliczenie` ma zawierać projekt zaliczeniowy, który można uruchomić i sprawdzić jak działa. Projekt korzysta z Wokwi i Platform IO, oraz zakłada wykorzystanie VS Code. Dokładnie ten sam zestaw narzędzi z którego korzystaliśmy na tych laboratoriach.

3. Raport PDF — Definition of Done

Raport PDF musi zawierać screenshoty całego okna IDE VS Code.

Na zrzutach ma być widoczne:

- kod Sensor Node,
- działająca lokalna symulacja Wokwi z widocznymi panelami czujników,
- Bridge COM/HTTP,
- reagująca konsola Klienta / Serwisu alarmowego C#,
- dla PERFORMANCE: zapis do `incidents.csv`,
- dla DELIGHTER: test `X-API-Key` oraz odpowiedź `401 Unauthorized` przy braku albo błędnym kluczu.

Screenshots mają być dowodem uruchomienia systemu. Wycięty fragment konsoli bez kontekstu nie wystarcza.

Wymagania dla poziomów trudności

BASIC: Obowiązkowy — 35 pkt

Use case: Operator systemu chce widzieć aktualny stan bezpieczeństwa i fizycznej integralności archiwum dowodowego. System różnicuje stopień zagrożenia: pojedyncza anomalia to zdarzenie (Security Event), natomiast jednoczesne naruszenie dwóch sensorów to incydent krytyczny (Critical Incident).

Wymaganie funkcjonalne: Sensor Node (C++) na ESP32 odczytuje dane z czujników BMP180 oraz PIR. Sformatowany payload JSON przechodzi przez Bridge COM/HTTP (C#) do aplikacji Client / Consumer (C#).

System implementuje dwa niezależne tryby operacyjne w oparciu o fuzję danych (data fusion):

1. **Tryb Monitorowania (Telemetry Path):** Stały podgląd na aktualne ciśnienie atmosferyczne oraz stan czujnika ruchu.
2. **Ścieżka Zdarzeń i Incydentów (Event & Incident Engine):**
 - **Security Event:** Sytuacja, w której *tylko jeden* z czujników przekroczył normę (wykryto ruch **LUB** nastąpił spadek ciśnienia o ustaloną deltę). Zdarzenie ma charakter efemeryczny – po upływie dokładnie 60 sekund od ustania anomalii musi automatycznie zniknąć z pamięci (TTL / Cache Eviction). Kolejne odpytanie po tym czasie zwraca stan pusty.
 - **Critical Incident:** Sytuacja, w której *oba czujniki* przekroczyły normę jednocześnie (Ruch = TRUE **AND** spadek ciśnienia). Status incydentu jest trwały – nie znika z pamięci po minucie i wymaga ręcznego skasowania/resetu.

Implementacja architektury i wybór metod HTTP (GET/POST) do przekazywania stanów między Bridge a Klientem leży całkowicie po stronie wykonawcy. Bridge nie implementuje logiki biznesowej.

Założenia techniczne (wspólne dla całego projektu):

- **Sensor Node:** C++, Arduino framework, ESP32 (lokalnie w VS Code przez wtyczkę Wokwi lub fizyczny hardware).
- **BMP180:** I2C (sprzętowe piny SDA/SCL).
- **PIR:** Digital In (wejście cyfrowe).
- **Transport z urządzenia:** USB lub Serial – za pomocą mechanizmu przekierowania portu szeregowego na strumień TCP (standard RFC 2217 / Wokwi Serial Port Forwarding).

- **Bridge COM/HTTP:** C#, aplikacja pośrednicząca COM/HTTP. Przekazuje surowe dane szeregowo na żądania HTTP, nie implementuje logiki biznesowej ani logiki bezpieczeństwa (chyba, że dane zadanie mówi inaczej).
- **Client / Consumer:** Aplikacja serwerowa C# (serwis alarmowy). To tutaj student implementuje całą logikę stanów, okna czasowe, walidację czasu Hosta oraz ewentualne sprawdzanie nagłówków.
- **Kontrakt danych:** Stała, z góry zdefiniowana struktura dokumentu JSON przesyłana z ESP32 (te same klucze i typy danych).

● PERFORMANCE: Opcjonalnie — dla osób celujących w 5 / bdb — 15 pkt

Use case: W archiwum dowodowego następuje potencjalne naruszenie strefy. System nie ma tylko pokazać danych bieżących. Serwis alarmowy musi samodzielnie przeprowadzić korelację danych w czasie, rozpoznać rzeczywiste zagrożenie, podnieść status systemu oraz odłożyć trwały ślad audytowy (audit trail).

Wymaganie funkcjonalne: Client / Serwis alarmowy C# stale analizuje strumień danych przychodzących z Bridge'a i przechodzi w stan alarmowy, generując komunikat:

[CRITICAL ANOMALY] FACILITY BREACH

Logika detekcji opiera się na rozróżnieniu czasu i trwałości anomalii:

1. **Weryfikacja trwałości (Incident Isolation):** Stan naruszenia (anomalía z sensora PIR = true LUB spadek ciśnienia z BMP180 poniżej normy / gwałtowny skok delty) zostaje zaklasyfikowany jako **Critical Incident** dopiero wtedy, gdy zmiana stanu **utrzymuje się nieprzerwanie przez ponad 60 sekund** (odfiltrowanie chwilowych wahań i szumów pomiarowych).
2. **Korelacja natychmiastowa:** Jeżeli *oba czujniki* przekroczą normę jednocześnie (ruch = true **AND** spadek ciśnienia), system pomija regulę 1 minuty i natychmiast zgłasza **Critical Incident**.
3. **Aktywacja alarmu (Facility Breach):** Stan **Critical Incident** zostaje zaklasyfikowany jako ostateczne naruszenie strefy **FACILITY BREACH** wyłącznie wtedy, gdy pojawi się w **symulowanych godzinach nocnych** (pomiędzy 22:00 a 06:00 czasu systemowego Hosta). W godzinach dziennych pozostaje raportowany jako **Critical Incident**.

Każdy przypadek zmiany stanu systemu na **Security Event**, **Critical Incident** lub **FACILITY BREACH** musi zostać niezwłocznie dopisany (Append) do pliku tekstowego: **incidents.csv**

Wiersz logu musi zawierać bezwzględny znacznik czasu UTC w standardzie ISO 8601 oraz szczegóły wyzwolenia.

● **DELIGHTER: Dla pasjonatów cybersecurity — bez punktów, dla siebie**

Use case: Administrator systemu wdraża mechanizmy utwardzania infrastruktury (security hardening) na poziomie sieciowym. Interfejs HTTP wystawiony przez Bridge COM/HTTP (C#) nie może być powszechnie dostępny dla każdego procesu w sieci lokalnej. System wymusza weryfikację tożsamości komponentów i podnosi alert w przypadku prób sabotażu czasu pracy serwisu.

Wymaganie funkcjonalne: Aplikacja Client / Consumer (C#) zabezpiecza swoje punkty końcowe i logiczne. Każde żądanie przesyłane pomiędzy komponentami infrastruktury (lub zewnętrzne zapytanie kontrolne) podlega rygorom kontroli dostępu oraz kontekstu czasu:

1. **Uwierzytelnianie Żądań (Request Authorization):** Komponenty sprawdzają obecność i poprawność dedykowanego nagłówka HTTP (HTTP Header): **X-API-Key: <TajneHasło>**
Brak nagłówka lub przekazanie nieprawidłowego tokenu skutkuje natychmiastowym przerwaniem obsługi żądania i zwróceniem kodu odpowiedzi **401 Unauthorized**.
2. **Analiza Ryzyka w Zespole Bezpieczeństwa (Red/Blue Lens):** Do raportu technicznego należy dołączyć zwięzłą, inżynierską analizę architektury (max 4–5 zdań) odpowiadającą na dwa wyzwania:
 - **Strategia Blue Team (Defensive):** Zaproponowanie minimum dwóch usprawnień, które wyeliminują podatność przesyłania klucza jawnym tekstem (plaintext) przez HTTP.
 - **Wektor Ataku Red Team (Offensive / Sabotage):** **Wektor Ataku Red Team (Offensive / Sabotage):** Zaproponowanie dwóch ogólnych scenariuszy naruszenia integralności danych lub sabotażu toru transmisyjnego (data tampering / denial of service), które pozwolą ukryć zmianę stanu faktycznego sensorów i wymuszają na centrali Client / Consumer ciągłe raportowanie stanu **NORMAL**.

Implementacja mechanizmów obronnych i walidacji nagłówków leży całkowicie po stronie wykonawcy i jest realizowana programistycznie w kodzie aplikacji C#.

Instrukcja demo: Wireshark + MQTT Awareness

1. MQTT — krótkie wprowadzenie

W systemach produkcyjnych IoT często stosuje się MQTT, bo pasuje do modelu zdarzeń i telemetry. Zamiast synchronicznego odpytywania HTTP, MQTT używa modelu **publish-subscribe**: urządzenie publikuje wiadomość do brokera, a odbiorcy subskrybują interesujące ich tematy. Broker rozdziela producentów danych od konsumentów danych, a narzut komunikacyjny jest zwykle mniejszy niż przy częstym HTTP polling. W LAB 07 MQTT jest tylko punktem orientacyjnym architektonicznym — projektu nie implementujemy w MQTT.

2. Wireshark — pokaz plaintext

Celem demo jest pokazanie faktu technicznego: zwykły HTTP nie ukrywa danych.

Kroki:

1. Uruchom lokalny Bridge / endpoint HTTP.
2. Uruchom Wireshark.
3. Wybierz interfejs pętli zwrotnej:
 - Loopback,
 - Localhost,
 - albo właściwy lokalny interfejs dostępny na stanowisku.
4. Rozpocznij przechwytywanie.
5. Wykonaj pojedyncze żądanie do Bridge'a, np. przez `curl` albo `.http`.
6. Użyj filtra, np.:
 - `http`,
 - `tcp.port == 5080`,
 - albo portu używanego przez lokalny Bridge.
7. Otwórz pakiet HTTP.

8. Wskaż:

- ścieżkę żądania,
- payload JSON,
- nagłówek **X-Api-Key**, jeżeli jest użyty,
- odpowiedź serwera.

Efekt WOW: w pakiecie widać, że nieszyfrowany payload JSON oraz klucze API lecą jawnym tekstem (plaintext).

Wniosek: **X-Api-Key** nie jest tym samym co HTTPS. Token może ograniczyć dostęp aplikacyjny, ale nie ukrywa danych przed obserwatorem ruchu.

Micro-Cheat Sheet dla studentów

1. UTC ISO 8601 w C#

Do zapisu czasu incydentu użyj UTC:

```
var timestampUtc = DateTime.UtcNow.ToString("O");
```

Format "O" daje zapis zgodny z ISO 8601 / round-trip. Nie zapisuj lokalnego czasu bez kontekstu, bo później nie będzie wiadomo, jak interpretować zdarzenie.

2. Dopisanie linii do pliku

Do dopisania rekordu do `incidents.csv` użyj:

```
File.AppendAllText("incidents.csv", line + Environment.NewLine);
```

Sens:

- jeśli plik istnieje — linia zostaje dopisana,
- jeśli plik nie istnieje — zostanie utworzony,
- poprzednie incydenty nie zostają nadpisane.

3. Minimalny rekord incydentu

Rekord ma pozwolić odpowiedzieć na cztery pytania:

- kiedy wystąpiło zdarzenie,
- jakie dane były podstawą decyzji,
- jaki był powód alarmu,
- jaki incydent został zapisany.

Przykładowe format logu audytowego

timestampUtc,pressure,motion,reason,event

Przykładowe logi audytowe

Stan normalny (Uruchomienie systemu / powrót do normy)

2026-05-19T22:01:00.000000Z,1013.25,false,SYSTEM_INITIALIZATION,NORMAL

Pojedyncza anomalia – ruch wykryty w dzień (Trwa krócej niż minutę)

2026-05-20T12:30:15.450000Z,1012.80,true,MOTION_DETECTED,SECURITY EVENT

Pojedyncza anomalia – spadek ciśnienia w dzień (Trwa krócej niż minutę)

2026-05-20T14:22:40.110000Z,995.10,false,PRESSURE_DROP,SECURITY EVENT

Stan stabilny po ustaniu chwilowych anomalii dziennych (Wyczyszczenie stanu po 60s)

2026-05-20T14:23:40.110000Z,1013.00,false,TIMEOUT_EXPIRED,NORMAL

Anomalia jednego czujnika trwająca nieprzerwanie ponad 60 sekund w dzień (Zmiana stanu na trwający)

2026-05-20T15:45:00.000000Z,1012.50,true,BREACH_EVENT,CRITICAL INCIDENT

Korelacja natychmiastowa – oba czujniki naraz w dzień (Brak czekania na minutę)

2026-05-20T16:10:05.120000Z,992.30,true,INCIDENT,CRITICAL INCIDENT

Naruszenie strefy w nocy – trwała anomalia jednego czujnika (Ponad 60 sekund w godzinach nocnych)

2026-05-20T23:14:20.050000Z,1012.40,true,BREACH_EVENT,FACILITY BREACH

```
# Naruszenie strefy w nocy – natychmiastowa korelacja obu czujników w godzinach nocnych  
2026-05-20T23:45:12.890000Z,989.15,true,INCIDENT,FACILITY BREACH
```

4. RFC 2217 i Wokwi Serial Port Forwarding

Wtyczka Wokwi przekierowuje dane `Serial` na port TCP (standard RFC 2217). W pliku `wokwi.toml` należy wskazać port sieciowy (np. `sim.serialPort = 4000`), z którym Bridge (C#) połączy się za pomocą adresu `127.0.0.1:4000` zamiast fizycznego portu `COM`.

Agenda zajęć

Część A: Sprawy Organizacyjne

- Przypomnienie Lab 5 oraz sprawdzenie indywidualnych postępów w ramach HW.
- Cel na Lab 6

Część B: Warsztat „Eksperymenty Pomiarowe: stabilność, opóźnienia i retry”

- **Intro:** Omówienie przejścia od samego uruchomienia endpointu do świadomej oceny zachowania systemu: urządzenie nie tylko raportuje stan przez **GET /status**, ale staje się punktem eksperymentu, w którym mierzymy czas odpowiedzi, liczbę błędów, skuteczność ponowień oraz zapisujemy dane w postaci **CSV** albo czytelnego logu.
- **Zadanie BASIC:** Uruchomienie znanego toru **BMP180 → ESP8266 → Serial → Bridge → HTTP**, potwierdzenie działania endpointu **/reading**, dołożenie wyświetlacza **LCD 2x16 I2C** jako lokalnej obserwowalności urządzenia, uporządkowanie połączeń na **płytce prototypowej**, wykonanie serii pomiarów czasu odpowiedzi endpointu, zapis wyników do **CSV/logu**, zastosowanie prostego **retry** ze stałym odstępem oraz przygotowanie **3 krótkich wniosków** opartych o dane.
- **Zadanie PERFORMANCE:** Rozszerzenie eksperymentu o krótki test przeciążeniowy (**burst**, np. 20 żądań/s przez 10 s), porównanie dwóch konfiguracji ponowień (**backoff 100 ms vs 500 ms**) oraz ocena wpływu tych ustawień na liczbę sukcesów, błędów, ponowień i typowy czas odpowiedzi.
- **Check-off & Audit Trail:** Weryfikacja działania endpointu **/reading** przez **curl** lub plik **.http**, potwierdzenie lokalnego statusu na dołożonym **LCD 2x16 (T/P, BMP: OK/ERR albo SENT: licznik)**, sprawdzenie pliku **CSV/log** z pomiarami **latencyMs**, **retryCount** i statusem odpowiedzi, omówienie **3 wniosków technicznych**, Push kodu na GitHub (tag **lab6**) i czyszczenie poświadczeń Git.

Artefakt: Działający lokalny tor telemetryczny oparty o **ESP8266, BMP180, Serial i Bridge .NET**, rozszerzony w tym laboratorium o **LCD 2x16 I2C** oraz uporządkowane połączenia na **płytce prototypowej**, który wystawia ostatni odczyt przez endpoint **/reading** w formacie **JSON** oraz posiada artefakt eksperymentalny w postaci **CSV/logu** z pomiarami czasu odpowiedzi, informacją o błędach, ponowieniach i krótkimi wnioskami technicznymi.

Uwaga: Ze względu na ograniczenia sprzętowe sali (brak kart Wi-Fi w komputerach laboratoryjnych) realizujemy efekt sieciowy przez lokalny Bridge `Serial` -> `HTTP`, zamiast bezpośredniej komunikacji sieciowej z ESP. Symulowanie sieci WiFi jest możliwe z wykorzystaniem Wokwi.

Projekt: Eksperymenty Pomiarowe (BMP180 + LCD + Bridge + /reading)

Motyw przewodni: W tym laboratorium wracamy do lokalnego toru telemetrycznego znanego z **LAB 04** i rozszerzamy go o lokalną obserwowalność na urządzeniu. Twoim celem jest uruchomienie stanowiska, w którym **ESP8266** odczytuje dane z czujnika **BMP180**, wysyła dane przez **Serial**, lokalny **Bridge .NET** wystawia je dalej przez **HTTP** jako endpoint **/reading**, a Ty w ramach zadania **BASIC** dokładasz wyświetlacz **LCD 2x16 I2C** i porządkujesz połączenia na **platce prototypowej**. Dopiero na tak przygotowanym torze wykonujesz eksperyment: mierzysz czas odpowiedzi endpointu, liczbę błędów, skuteczność ponowień i zapisujesz dane w postaci **CSV** albo czytelnego logu.

Uwaga techniczna: Ograniczenie środowiskowe nadal obowiązuje. Komputery laboratoryjne nie posiadają kart Wi-Fi, dlatego nie budujemy na zajęciach scenariusza zależnego od pełnej komunikacji sieciowej pomiędzy stanowiskiem a fizycznym ESP. Korzystamy z ustalonego wcześniej obejścia i nie rozpisujemy go tutaj od zera ponownie — traktuj je jako obowiązujący kontekst warsztatowy z **LAB 04**. Główny scenariusz tego laboratorium jest przygotowany pod wersję sprzętową: **Wemos D1 + BMP180 + LCD 2x16 I2C + Serial + Bridge .NET + /reading**.

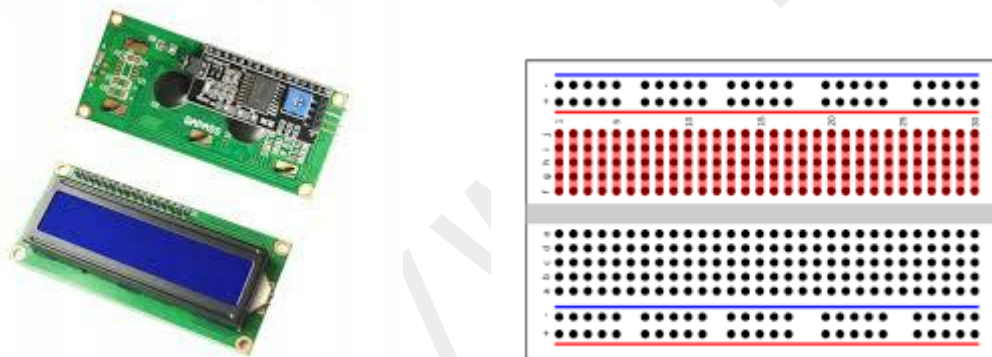
Ścieżka alternatywna dla chętnych: Jeżeli chcesz, możesz opracować wariant oparty wyłącznie o **symulator Wokwi** i komunikację poprzez wirtualnego **Wi-Fi**. Wtedy nie korzystasz z Bridge'a, tylko uruchamiasz wariant symulacyjny, w którym wirtualna płytka wystawia dane przez sieć. Szczegóły przygotowania symulacji Wi-Fi w Wokwi oraz ograniczenia wersji personal zostaną opisane w **Cheat Sheet**. To jest alternatywne środowisko uruchomieniowe, a nie zmiana głównego scenariusza zajęć.

Uwaga organizacyjna: W tym laboratorium dokładamy do stanowiska nowy element wyjściowy — wyświetlacz **LCD 2x16 I2C** — oraz porządkujemy połączenia przez **platkę prototypową**.

🧠 Zanim zaczniesz: Twój Hardware na dziś (Wemos D1 + BMP180 + LCD 2x16 I2C)

W ramach **LAB 06** punktem startowym jest tor telemetryczny z **LAB 04**: **BMP180** → **ESP8266** → **Serial** → **Bridge** → **HTTP**. Nie traktuj jednak wyświetlacza **LCD 2x16** jako gotowej części tego toru. LCD dopiero podłączasz w tym laboratorium i wykorzystujesz go jako prostą lokalną obserwowalność urządzenia.

W poprzednich laboratoriach budowałeś ścieżkę telemetryczną oraz kontrakt danych. Teraz zaczynasz mierzyć nie tylko wartość temperatury i ciśnienia, ale również zachowanie całego toru od urządzenia do endpointu **/reading**. **LCD** pomaga Ci lokalnie zobaczyć, czy urządzenie żyje i czy pomiar ma sens. **CSV/log** pokazuje natomiast, jak stabilnie odpowiada warstwa HTTP.



Inżynierskie „Dlaczego?”: System pomiarowy nie kończy się na tym, że czujnik zwróci liczbę. W praktyce musisz wiedzieć, czy dane są dostępne lokalnie, czy przechodzą przez transport, czy endpoint odpowiada oraz czy odpowiedzi są stabilne w czasie. **LCD** daje Ci prostą lokalną obserwowalność na urządzeniu, **Serial** przenosi rekord pomiarowy do komputera, **Bridge** wystawia go przez **HTTP**, a klient pomiarowy sprawdza, jak ten tor zachowuje się w serii wywołań. To jest przejście od „działa” do „wiem, jak działa”.

1. Schemat połączeń (Pinout Mapping)

W tym laboratorium używasz **BMP180** oraz dokładasz **LCD 2x16 I2C**. Oba moduły pracują na magistrali **I2C**, dlatego współdzielą te same linie **SDA** i **SCL**. To nie jest konflikt pinów. To jest normalna praca magistrali I2C. Warunek jest jeden: urządzenia muszą mieć różne adresy I2C.

BMP180 → Wemos D1

Podłącz czujnik **BMP180** do płytki **Wemos D1** dokładnie według poniższej tabeli:

BMP180 (Sensor)	Wemos D1 (Board)	Funkcja / Opis
VIN	3.3V	⚠ Zasilanie sensora. Nie podłączaj do 5V ani do VIN na płytce.
GND	GND	Masa układu (wspólny potencjał).
SDA	D4 (GPIO4)	Linia danych magistrali I2C.
SCL	D3 (GPIO5)	Linia zegarowa magistrali I2C.

Krytyczna pułapka: Na module **BMP180** pin **VIN** oznacza zasilanie sensora pracującego w standardzie **3.3V**. Na płytce **Wemos D1** pin **VIN** przenosi zasilanie z USB. To nie jest to samo. Błędne podłączenie sensora do niewłaściwego napięcia może skończyć się uszkodzeniem modułu.

LCD 2x16 I2C → Wemos D1

W ramach zadania **BASIC** dołącz wyświetlacz **LCD 2x16 I2C** do tej samej magistrali **I2C**, zgodnie z poniższą tabelą:

LCD 2x16 I2C	Wemos D1 (Board)	Funkcja / Opis
VCC	5V	Zasilanie modułu LCD z adapterem I2C.
GND	GND	Masa układu (wspólny potencjał).
SDA	D4 (GPIO4)	Linia danych magistrali I2C — współdzielona z BMP180.
SCL	D3 (GPIO5)	Linia zegarowa magistrali I2C — współdzielona z BMP180.

Uwaga praktyczna: To, że **BMP180** i **LCD** korzystają z tych samych linii **SDA** i **SCL**, nie oznacza błędu w połączeniu. I2C jest magistralą, a nie połączeniem „jeden pin — jedno urządzenie”. Właśnie dlatego płytka prototypowa jest tutaj potrzebna: pozwala rozdzielić zasilanie, masę oraz wspólne linie magistrali w sposób czytelny i stabilny.

Płytki prototypowa — jak ją traktować w tym labie

Płytki prototypowa nie jest osobnym „gadżetem”. Jest narzędziem porządkującym stanowisko i pokazuje w praktyce, dlaczego przy więcej niż jednym module trudno pracować wyłącznie na pojedynczych przewodach Dupont.

Element	Co robisz na breadboardzie	Po co
GND	Rozprowadzasz wspólną masę do BMP180 i LCD.	Wszystkie moduły muszą mieć wspólny potencjał odniesienia.
3.3V	Doprowadzasz zasilanie do BMP180.	BMP180 nie podłączasz do 5V.
5V	Doprowadzasz zasilanie do LCD 2x16 I2C.	LCD z adapterem I2C zwykle pracuje z 5V.
SDA	Łączysz D4/GPIO4 z SDA BMP180 i SDA LCD.	Wspólna linia danych I2C.
SCL	Łączysz D3/GPIO5 z SCL BMP180 i SCL LCD.	Wspólna linia zegarowa I2C.

Uwaga praktyczna: Nie mieszaj zasilania **3.3V** i **5V**. To nie są „dwa plusy do wyboru”. **BMP180** podłączasz do **3.3V**, a **LCD 2x16 I2C** do **5V**. Masa jest wspólna.

2. Skąd wziąć kod i narzędzia do LAB 06

Na te zajęcia nie budujesz całego toru telemetrycznego od zera. Korzystasz z architektury znanej z **LAB 04** i rozszerzasz ją o lokalny wyświetlacz oraz eksperyment pomiarowy. Kody źródłowe są dostępne na MS Teams w sekcji “Praca na zajęciach”.

Przygotuj w **PlatformIO / VS Code** projekt dla **ESP8266**, w którym:

- uruchomisz **Serial** do wysyłania rekordów pomiarowych,
- odczytasz temperaturę i ciśnienie z **BMP180**,
- dołożysz obsługę **LCD 2x16 I2C** jako lokalnej obserwowalności,
- wyślesz jedną linię **JSON** przez port szeregowy,
- uruchomisz lokalny **Bridge .NET** wystawiający endpoint **/reading**,
- wykonasz pomiary endpointu po stronie klienta.

Do testowania endpointów używamy tak samo jak wcześniej:

- **curl** z terminala,
- albo pliku **.http** w VS Code.

Do właściwego eksperymentu użyjesz klienta pomiarowego:

- aplikacji konsolowej **.NET**,
- albo prostszego wariantu **PowerShell / BAT** na Windows 10, jeżeli nie czujesz się jeszcze pewnie w C#.

Twoim zadaniem na tych zajęciach nie jest budowa wielkiej architektury, tylko wykonanie czegoś znacznie ważniejszego dydaktycznie: rozszerzenie znanego toru o **lokalną obserwowalność**, a następnie wykonanie pomiaru **latency**, zapisu **CSV/logu**, prostego **retry** i krótkich wniosków technicznych.

3. Co dokładnie robi każdy element systemu

Żeby nie zgubić architektury, zapamiętaj prosty podział odpowiedzialności:

- **BMP180** – mierzy temperaturę i ciśnienie,
- **LCD 2x16 I2C** – zostaje dołożony w tym laboratorium i pokazuje lokalny stan urządzenia: pomiar, status sensora albo licznik wysłanych rekordów,
- **ESP8266** – odczytuje dane z BMP180, aktualizuje LCD, loguje przez **Serial** i wysyła rekord JSON,
- **Bridge .NET** – czyta dane z portu COM, rozpoznaje JSON lub błąd i wystawia lokalne API HTTP,
- **Klient HTTP (curl / .http / klient .NET / PowerShell)** – odpytuje endpoint **/reading**, mierzy czas odpowiedzi i zapisuje wyniki do **CSV** albo logu.

Złota zasada: jeżeli coś nie działa, nie zgaduj. Najpierw ustal, na którym odcinku psuje się tor danych:

1. sensor / okablowanie,
2. LCD / lokalna obserwowalność,
3. Serial / rekord JSON,
4. Bridge / HTTP,
5. klient pomiarowy / CSV.

4. Ścieżka alternatywna dla chętnych – Wokwi i Wi-Fi

Jeżeli chcesz, możesz opracować wariant oparty wyłącznie o **symulator Wokwi** i komunikację sieciową przez **Wi-Fi**, wykorzystując wiedzę zdobytą w **LAB 02** i **LAB 03**. Nie jest to główny scenariusz zajęć. Główny opis laboratorium dotyczy wersji sprzętowej: **Wemos D1 + BMP180 + LCD + Serial + Bridge + /reading**.

W wariancie symulacyjnym możesz potraktować wirtualną płytkę jako alternatywne środowisko uruchomieniowe i samodzielnie odtworzyć logikę:

- odczytu danych pomiarowych,
- wystawienia endpointu przez Wi-Fi,
- testowania endpointu narzędziowo,
- wykonania serii pomiarów latencji,
- zapisu wyników do CSV albo logu.

Szczegółowy opis przygotowania Wi-Fi w **Wokwi**, wraz z ograniczeniami wersji personal, znajdziesz w **Cheat Sheet**. Tutaj zapamiętaj tylko jedną rzecz: symulator jest alternatywą do pracy domowej i eksperymentów poza salą, a nie zastępstwem głównego scenariusza hardware'owego.

Setup Środowiska: VS Code + PlatformIO + .NET + curl

Pracujemy nadal w **VS Code**, ale łączymy dwa światy: **Embedded** i **backend lokalny**. Twoim celem jest uruchomienie znanego toru **ESP8266** → **Serial** → **Bridge** → **HTTP**, dołożenie **LCD 2x16 I2C** jako lokalnej obserwowalności, a następnie wykonanie eksperymentu na endpointcie **/reading**.

1. Rozszerzenia i narzędzia

Na tych zajęciach pracujemy w **VS Code**. Przygotuj środowisko tak, abyś mógł kompilować firmware, monitorować **Serial**, uruchamiać Bridge'a i testować HTTP bez walki z narzędziami.

- Upewnij się, że masz działające **PlatformIO** w VS Code.
- Sprawdź, czy możesz wgrywać firmware na **ESP8266**.
- Upewnij się, że działa **Serial Monitor** z poprawnym **baud rate**.
- Upewnij się, że masz dostęp do **dotnet CLI** z poziomu terminala.
- Do testowania endpointów używamy:
 - **curl** z terminala,
 - albo pliku **.http** w VS Code.

Złota zasada: zanim zaczniesz mierzyć latency endpointu, najpierw brutalnie sprawdź, czy urządzenie naprawdę widzi czujnik, czy dołożony LCD pokazuje sensowny stan i czy Bridge naprawdę wystawia **/reading**.

2. Projekt firmware dla ESP8266

Punktem wyjścia jest projekt **PlatformIO** dla **Wemos D1 / ESP8266** z obsługą **BMP180** i **Serial**. W tym laboratorium rozszerzasz go o obsługę **LCD 2x16 I2C**.

Twój program ma wykonać pięć rzeczy:

1. uruchomić **Serial**,
2. uruchomić magistralę **I2C**,
3. odczytać temperaturę i ciśnienie z **BMP180**,
4. pokazać lokalny status na dołożonym **LCD 2x16**,
5. wysłać przez **Serial** jedną linię **JSON** z aktualnym pomiarem.

Przykładowe informacje na LCD:

T: 23.4 C
P: 1012 hPa

albo:

BMP: OK
SENT: 42

Uwaga: Na tym etapie nie chodzi o „ładny panel operatorski”. Chodzi o lokalną obserwowalność. LCD ma pomóc szybko zobaczyć, czy urządzenie żyje i czy pomiar ma sens. Pełny artefakt eksperymentalny nadal powstaje po stronie klienta jako **CSV/log**.

3. Diagnostyka przez Serial i LCD

Zanim wykonasz eksperyment przez HTTP, najpierw musisz mieć lokalny dowód, że system widzi to, co dzieje się na czujniku i że dołożony wyświetlacz nie jest tylko ozdobą.

W **Serial Monitorze** powinieneś umieć zobaczyć co najmniej:

- poprawną linię **JSON** z temperaturą i ciśnieniem,
- komunikat błędu, jeżeli **BMP180** nie odpowiada,
- licznik albo znacznik czasu kolejnych rekordów.

Na **LCD 2x16** powinieneś umieć zobaczyć co najmniej:

- aktualną temperaturę,
- aktualne ciśnienie,
- albo prosty status **BMP: OK / BMP: ERR**.

Po co to robimy?

Bo HTTP ma być kolejną warstwą obserwowalności, nie pierwszą. Jeżeli czujnik nie działa, LCD pokazuje błąd albo **Serial** nie wysyła poprawnego JSON-a, endpoint **/reading** nie naprawi Ci rzeczywistości. Bridge może tylko wystawić to, co dostał albo czego nie dostał.

4. Testowanie endpointu: curl / .http

Zanim uruchomisz klienta pomiarowego, najpierw sprawdź endpoint „na surowo”. To jest obowiązkowy etap diagnostyczny.

Podstawowy endpoint tego laboratorium:

- **/reading**

Test przez **curl**:

```
curl http://127.10.10.10:5080/reading
```

Możesz też przygotować własny plik **.http** w VS Code i wykonywać zapytania bezpośrednio z edytora.

Po co to robimy?

Bo jeśli **/reading** zwraca błąd, puste dane albo odpowiedź niezgodną z kontraktem, musisz to zobaczyć od razu na poziomie HTTP, a nie dopiero przy pomiarze latency, retry albo generowaniu CSV.

5. Kontrakt `/reading` — co ma zwracać system

W tym laboratorium endpoint `/reading` nie może być atrapą. Ma zwracać prawdę o ostatnim poprawnym odczycie z toru **BMP180** → **ESP8266** → **Serial** → **Bridge** → **HTTP**.

Minimalnie odpowiedź powinna zawierać informacje takie jak:

- temperatura,
- ciśnienie,
- jednostki,
- czas ostatniego odczytu,
- informacja o ostatnim błędzie.

Przykładowa odpowiedź powinna wyglądać (lub być zbliżona do):

```
{
  "temperature": 23.41,
  "pressureHpa": 1011.92,
  "unitTemperature": "C",
  "unitPressure": "hPa",
  "lastUpdateUtc": "2026-04-08T10:15:30Z",
  "lastError": null
}
```

6. Audit środowiska przed startem

Zanim przejdziesz dalej, sprawdź trzy rzeczy:

- czy firmware kompiluje się i wgrywa bez błędu,
- czy dołożony **LCD 2x16** pokazuje lokalny stan urządzenia,
- czy endpoint **/reading** odpowiada przez **curl** lub **.http**.

Środowisko uznajemy za gotowe dopiero wtedy, gdy:

1. urządzenie startuje poprawnie,
2. LCD pokazuje sensowny status pomiaru,
3. **Serial** wysyła poprawny JSON albo czytelny błąd,
4. Bridge wystartował,
5. HTTP odpowiada i zwraca dane zgodne z kontraktem.

Hardware / Runtime Audit: Tak jak w poprzednich laboratoriach błędny kabel albo zła konfiguracja portu potrafiły zablokować cały tor danych, tak tutaj źle podłączony sensor, źle rozprowadzona magistrala I2C, pomyłone zasilanie albo zajęty port COM potrafią zablokować cały sens ćwiczenia. Traktuj konfigurację hardware'u, firmware'u, Bridge'a i klienta jako integralną część systemu, a nie „drobiazg techniczny”.

● BASIC: (MUST - Wymagane)

Cel: Rozszerzenie znanego toru telemetrycznego **BMP180** → **ESP8266** → **Serial** → **Bridge** → **HTTP** o lokalną obserwowalność na **LCD 2x16 I2C**, uporządkowanie połączeń na **platce prototypowej** oraz wykonanie podstawowego eksperymentu pomiarowego na endpointcie **/reading**: czas odpowiedzi, status odpowiedzi, retry, **CSV/log** i 3 krótkie wnioski techniczne.

Zadanie A: Podłączenie LCD 2x16 I2C i uporządkowanie układu

Cel: Dołożenie wyświetlacza **LCD 2x16 I2C** do znanego układu z **BMP180** tak, aby urządzenie pokazywało lokalny stan pomiaru, a połączenia były czytelnie rozprowadzone przez płytkę prototypową.

- **Akcja 1 (Hardware Layout):** Podłącz **BMP180** i **LCD 2x16 I2C** do płytki **Wemos D1** zgodnie z mapowaniem z sekcji *Zanim zaczniesz*.
- **Akcja 2 (Breadboard Layout):** Rozprowadź na płytce prototypowej wspólną masę, zasilanie **3.3V** dla **BMP180**, zasilanie **5V** dla **LCD** oraz wspólne linie **SDA/SCL** magistrali **I2C**.
- **Akcja 3 (LCD Smoke Test):** Uruchom prosty komunikat startowy na LCD, np. **LAB06 / BMP READY**.
- **Akcja 4 (Local Status):** Wyświetl na LCD lokalny stan pomiaru, np. temperaturę i ciśnienie albo status **BMP: OK/ERR** oraz licznik wysłanych rekordów.
- **DoD (Definition of Done):**
 - ☐ **BMP180** jest poprawnie podłączony do **Wemos D1**.
 - ☐ **LCD 2x16 I2C** działa i pokazuje prosty komunikat.
 - ☐ Płytkę prototypową porządkuje zasilanie, masę oraz linie **I2C**.
 - ☐ LCD pokazuje lokalny stan urządzenia, a nie przypadkowy tekst testowy.

Zadanie B: Potwierdzenie toru /reading

Cel: Potwierdzenie, że znany tor **BMP180** → **ESP8266** → **Serial** → **Bridge** → **HTTP** działa poprawnie i że endpoint **/reading** zwraca aktualny kontrakt danych pomiarowych.

- **Akcja 1 (Serial Probe):** Sprawdź w **Serial Monitorze**, czy ESP8266 wysyła poprawną linię **JSON** z temperaturą i ciśnieniem albo czytelny komunikat błędu.
- **Akcja 2 (Bridge Start):** Uruchom lokalny **Bridge .NET** z poprawnym portem **COM** i prędkością transmisji.
- **Akcja 3 (HTTP Smoke Test):** Przetestuj endpoint **/reading** przez **curl** albo plik **.http**.
- **Akcja 4 (Contract Check):** Sprawdź, czy odpowiedź zawiera dane pomiarowe i czy jest zgodna z tym, co wysyła urządzenie przez **Serial**.
- **DoD (Definition of Done):**
 - ☐ **Serial** pokazuje poprawny rekord pomiarowy albo czytelny błąd.
 - ☐ Bridge startuje i odbiera dane z portu **COM**.
 - ☐ Endpoint **/reading** odpowiada przez HTTP.
 - ☐ Odpowiedź **JSON** zawiera temperaturę, ciśnienie oraz informacje diagnostyczne przewidziane w kontrakcie.

Zadanie C: Pomiar czasu odpowiedzi endpointu **/reading**

Cel: Wykonanie podstawowego eksperymentu pomiarowego: sprawdzenie, jak szybko i jak stabilnie odpowiada endpoint **/reading** w serii wywołań.

- **Akcja 1 (Measurement Client):** Przygotuj klienta pomiarowego w **.NET Console**, który wykonuje serię wywołań endpointu **/reading**. Jeżeli nie czujesz się jeszcze pewnie w C#, możesz przygotować prostszy wariant w **PowerShell / BAT** na Windows 10.
- **Akcja 2 (Latency Measurement):** Zmierz czas odpowiedzi każdego requestu jako **latencyMs**.
- **Akcja 3 (Result Logging):** Zapisz wynik każdej próby do **CSV** albo czytelnego logu.
- **Akcja 4 (Sample Size):** Wykonaj serię **20–30 requestów**, bez ręcznego wybierania tylko udanych wyników.
- **DoD (Definition of Done):**
 - ☐ Klient wykonuje serię requestów do **/reading**.
 - ☐ Każdy pomiar zawiera **latencyMs**.
 - ☐ Wyniki są zapisane do **CSV/logu**.
 - ☐ Seria zawiera minimum **20–30 prób**.

Zadanie D: Retry ze stałym odstępem

Cel: Dodanie prostego mechanizmu ponowienia próby, aby sprawdzić, czy błąd pojedynczego requestu jest trwały, czy chwilowy.

- **Akcja 1 (Failure Detection):** Rozpoznaj sytuację błędu: timeout, wyjątek transportowy albo odpowiedź HTTP spoza zakresu sukcesu.
- **Akcja 2 (Single Retry):** Dodaj jedno ponowienie requestu po krótkim, stałym odstępie.
- **Akcja 3 (Retry Logging):** Zapisz do **CSV/logu**, czy dana próba wymagała retry oraz ile ponowień wykonano.
- **Akcja 4 (Result Interpretation):** Odróżnij sukces za pierwszym razem od sukcesu dopiero po ponowieniu.
- **DoD (Definition of Done):**
 - ☐ Klient wykonuje retry po błędzie albo timeoutcie.
 - ☐ W logu widoczny jest **retryCount**.
 - ☐ Wynik końcowy próby ma oznaczenie sukcesu albo błędu.
 - ☐ Potrafisz wskazać, które odpowiedzi były stabilne, a które wymagały ponowienia.

Zadanie E: CSV/log i 3 wnioski techniczne

Cel: Zamknięcie eksperymentu krótką analizą danych, a nie samym uruchomieniem kodu.

- **Akcja 1 (CSV Format):** Zadbaj, aby rekord pomiarowy zawierał co najmniej: `timestamp`, `endpoint`, `attempt`, `statusCode`, `latencyMs`, `retryCount`, `success`, `error`.
- **Akcja 2 (Data Review):** Przejrzyj wyniki i sprawdź, czy widać typowy czas odpowiedzi, błędy albo ponowienia.
- **Akcja 3 (Three Conclusions):** Przygotuj **3 krótkie wnioski techniczne** oparte o dane z **CSV/logu**.
- **Akcja 4 (Audit Trail):** Zachowaj plik pomiarowy razem z kodem jako artefakt laboratoryjny.
- **DoD (Definition of Done):**
 - ☐ Masz plik **CSV/log** z wynikami pomiarów.
 - ☐ Każdy rekord zawiera czas odpowiedzi i status próby.
 - ☐ Wnioski odnoszą się do danych, a nie do samego wrażenia z testu.
 - ☐ Artefakt pomiarowy można pokazać podczas check-off.

● PERFORMANCE: (dla szybszych / po zajęciach)

Cel: Rozszerzenie eksperymentu o krótkie obciążenie oraz porównanie dwóch konfiguracji ponowień, aby zobaczyć wpływ ustawień retry/backoff na stabilność i czas odpowiedzi endpointu **/reading**.

Zadanie A: Krótki test burst

Cel: Sprawdzenie, jak endpoint **/reading** zachowuje się przy serii szybkich wywołań.

- **Akcja 1 (Burst Setup):** Przygotuj wariant klienta, który wykonuje krótką serię szybkich requestów, np. około **20 req/s przez 10 s**.
- **Akcja 2 (Burst Logging):** Zapisz wyniki do **CSV/logu** w tym samym formacie co w BASIC.
- **Akcja 3 (Error Count):** Policz liczbę sukcesów, błędów i ponowień.
- **Akcja 4 (Latency Review):** Sprawdź, czy czasy odpowiedzi są podobne do pomiaru BASIC, czy zaczynają się rozjeżdżać.
- **DoD (Definition of Done):**
 - ☐ Test burst wykonuje serię szybkich wywołań endpointu.
 - ☐ Wyniki są zapisane do **CSV/logu**.
 - ☐ Potrafisz wskazać liczbę sukcesów, błędów i retry.
 - ☐ Potrafisz porównać zachowanie endpointu w BASIC i przy krótkim obciążeniu.

Zadanie B: Porównanie backoff 100 ms vs 500 ms

Cel: Porównanie dwóch ustawień odstępu przed retry i ocena, jak wpływają na wynik eksperymentu.

- **Akcja 1 (Config A):** Uruchom pomiar z backoffem **100 ms**.
- **Akcja 2 (Config B):** Uruchom pomiar z backoffem **500 ms**.
- **Akcja 3 (Comparison Table):** Porównaj liczbę sukcesów, błędów, ponowień oraz typowy i najgorszy czas odpowiedzi.
- **Akcja 4 (Trade-off):** Zapisz krótki wniosek, który wariant zachował się lepiej w Twoim teście i jaką cenę za to płaci.
- **DoD (Definition of Done):**
 - ☐ Masz dwa zestawy danych: **100 ms** i **500 ms**.
 - ☐ Porównujesz wyniki na podstawie zapisanych danych.
 - ☐ Widzisz różnicę między szybkim ponowieniem a spokojniejszym odstępem.
 - ☐ Wniosek wynika z pomiaru, a nie z założenia.

● DELIGHTERS (dla ambitnych)

Cel: Rozwinięcie eksperymentu w stronę innego modelu komunikacji i sprawdzenie, jak zmienia się sposób myślenia o danych, gdy wychodzisz poza klasyczny model HTTP request/response.

Zadanie A: UDP telemetria i numer sekwencyjny pakietu

Cel: Przygotowanie prostego wariantu telemetrii UDP, w którym dane są wysyłane jako pakiety z numerem sekwencyjnym.

- **Akcja 1 (UDP Sender):** Przygotuj prosty nadajnik UDP wysyłający rekord telemetryczny z numerem sekwencyjnym.
- **Akcja 2 (UDP Receiver):** Przygotuj odbiornik, który zapisuje odebrane pakiety do logu.
- **Akcja 3 (Sequence Number):** Dodaj numer pakietu, aby można było sprawdzić, czy coś zginęło po drodze.
- **Akcja 4 (Log Format):** Zapisz numer pakietu, czas odbioru i podstawowe dane telemetryczne.
- **DoD (Definition of Done):**
 - ☐ Pakiety UDP są wysyłane i odbierane.
 - ☐ Każdy pakiet ma numer sekwencyjny.
 - ☐ Log pozwala sprawdzić kolejność pakietów.
 - ☐ Potrafisz wskazać różnicę między HTTP request/response a UDP.

Zadanie B: Straty i jitter

Cel: Wykorzystanie logu UDP do prostej analizy strat pakietów i zmienności odstępów między odbiorami.

- **Akcja 1 (Loss Check):** Sprawdź, czy w numerach sekwencyjnych są przerwy.
- **Akcja 2 (Inter-arrival Time):** Policz odstęp czasu między kolejnymi odebranymi pakietami.
- **Akcja 3 (Jitter Estimate):** Oszacuj, czy odstęp są stabilne, czy zmienne.
- **Akcja 4 (Short Reflection):** Zapisz krótki komentarz, co w UDP jest łatwiejsze, a co trudniejsze niż w HTTP.
- **DoD (Definition of Done):**
 - ☐ Potrafisz policzyć utracone pakiety albo uzasadnić, że w próbie ich nie było.
 - ☐ Potrafisz pokazać odstęp między odebranymi pakietami.
 - ☐ Potrafisz wyjaśnić pojęcie jitter na podstawie własnego logu.
 - ☐ Potrafisz porównać ten wariant z pomiarem endpointu **/reading**.

⚡ Cheat Sheet

I. Architektura LAB 06: BMP180 + LCD + Serial → Bridge → HTTP → pomiar

W LAB 04 pracowałeś na lokalnym torze telemetrycznym:

```
BMP180 + ESP8266
    |
    | Serial / COM
    v
Bridge .NET
    |
    | GET /reading
    v
Klient HTTP / .NET / PowerShell
```

W LAB 06 wracamy do tego toru i dokładamy dwa elementy:

- **LCD 2x16 I2C** – lokalny podgląd stanu pomiaru na stanowisku,
- **pomiar klienta** – seria requestów do `/reading`, zapis `latencyMs`, `retryCount`, statusu odpowiedzi i błędów.

Najważniejsze warstwy:

- **BMP180** – źródło temperatury i ciśnienia,
- **ESP8266** – odczytuje sensor, pokazuje lokalny status na LCD i wysyła JSON przez `Serial`,
- **Bridge .NET** – czyta port COM i wystawia endpoint HTTP,
- **klient pomiarowy** – odpytuje `/reading`, mierzy czas odpowiedzi i zapisuje wynik do CSV/logu.

W tym laboratorium interesuje Cię nie tylko to, czy system zwraca dane, ale też **jak zachowuje się w serii wywołań**: czy odpowiada szybko, czy zwraca błędy, czy retry pomaga i czy wynik da się opisać na podstawie danych.

II. Płytki prototypowa — po co jest i jak działa

Płytki prototypowa, czyli **breadboard**, pozwala połączyć kilka modułów bez lutowania. W tym labie używasz jej do uporządkowania połączeń między **Wemos D1**, **BMP180** i **LCD 2x16 I2C**.

Najważniejsza zasada: nie wszystkie otwory są osobne.

Typowa płytka ma dwa rodzaje połączeń:

- **rzędy robocze** – krótkie grupy otworów połączone wewnątrz płytki,
- **szyny zasilania** – długie linie po bokach, zwykle oznaczone jako **+** i **-**.

W praktyce:

- do jednej szyny możesz doprowadzić **GND** i rozprowadzić masę do kilku modułów,
- osobno możesz rozprowadzić **3.3V** dla BMP180,
- osobno możesz rozprowadzić **5V** dla LCD, jeżeli używany moduł LCD tego wymaga,
- linie **SDA** i **SCL** możesz rozdzielić do dwóch urządzeń I2C: BMP180 i LCD.

Przykładowy sens rozprowadzenia:

Sygnał	Skąd	Dokąd	Uwaga
GND	Wemos D1	BMP180 + LCD	Masa musi być wspólna.
3.3V	Wemos D1	BMP180	BMP180 nie podłączaj do 5V.
5V	Wemos D1	LCD I2C	Zależnie od modułu LCD. Sprawdź opis płytki.
SDA	D4 / GPIO4	BMP180 SDA + LCD SDA	Wspólna linia danych I2C.
SCL	D3 / GPIO5	BMP180 SCL + LCD SCL	Wspólna linia zegara I2C.

Typowe błędy:

- moduł włożony w inne rzędy niż przewód,
- masa nie jest wspólna,
- 3.3V i 5V są połączone przypadkiem,
- SDA i SCL są zamienione miejscami,
- szyna zasilania po jednej stronie płytki nie jest połączona z drugą stroną.

Jeżeli układ „prawie działa”, ale czasem znika LCD albo sensor, sprawdź najpierw breadboard: masa, zasilanie, SDA, SCL, kontakt przewodów.

III. I2C — jedna magistrala, kilka urządzeń

BMP180 i **LCD 2x16 I2C** korzystają z tej samej magistrali **I2C**. To oznacza, że oba moduły mogą być podłączone do tych samych linii:

- **SDA** – dane,
- **SCL** – zegar.

To nie jest konflikt pinów. W I2C wiele urządzeń może pracować na jednej magistrali, jeżeli mają różne adresy.

W tym labie używasz mapowania znanego z wcześniejszego toru:

Linia I2C	Wemos D1 / ESP8266
SDA	D4 / GPIO4
SCL	D3 / GPIO5

Na ESP8266 Arduino Core piny dla I2C można jawnie ustawić przez `Wire.begin(int sda, int scl)`, a bez wskazania pinów domyślne są GPIO4 jako SDA i GPIO5 jako SCL. (arduino-esp8266.readthedocs.io)

Fragment inicjalizacji magistrali:

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(4, 5); // SDA = GPIO4, SCL = GPIO5
}
```

Adresy urządzeń:

- **BMP180** zwykle pracuje pod adresem **0x77**,
- **LCD 2x16 I2C** często pracuje pod adresem **0x27** albo **0x3F**.

Jeżeli LCD nie reaguje, a przewody są poprawne, sprawdź adres I2C. Krótki skaner I2C jest dobrym narzędziem diagnostycznym.

Szkic idei skanowania adresów:

```
for (byte address = 1; address < 127; address++) {  
  Wire.beginTransaction(address);  
  byte error = Wire.endTransmission();  
  
  if (error == 0) {  
    Serial.print("I2C device: 0x");  
    Serial.println(address, HEX);  
  }  
}
```

Wynik skanera powinien pokazać urządzenia obecne na magistrali. Jeżeli widzisz tylko jeden adres, a powinny być dwa, sprawdź zasilanie, masę, SDA i SCL.

IV. LCD 2x16 I2C — jak go traktować w LAB 06

LCD 2x16 pokazuje dwie linie po 16 znaków. W tym labie wystarczy lokalna informacja, która pomaga szybko ocenić stan urządzenia.

Sensowne warianty treści:

T: 23.4 C

P: 1012 hPa

albo:

BMP: OK

SENT: 42

albo przy błędzie:

BMP: ERR

CHECK I2C

Do wyświetlaczy LCD przez I2C można użyć biblioteki `LiquidCrystal_I2C`; dokumentacja Arduino opisuje ją jako bibliotekę do sterowania wyświetlaczami I2C funkcjami podobnymi do standardowego `LiquidCrystal`. ([Arduino Documentation](#))

Minimalny szkic pracy z LCD:

```
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("LAB06");
  lcd.setCursor(0, 1);
  lcd.print("BMP READY");
}
```

Jeżeli ekran świeci, ale nie pokazuje znaków:

- sprawdź kontrast potencjometrem na adapterze I2C,
- sprawdź adres `0x27` / `0x3F`,
- sprawdź zasilanie,
- sprawdź, czy `lcd.backlight()` jest wywołane,
- sprawdź, czy program rzeczywiście dochodzi do kodu LCD.

Przy aktualizacji wartości czyścić tylko potrzebny fragment albo nadpisywać stałą szerokością. LCD nie usuwa automatycznie resztek dłuższego tekstu.

Przykład problemu:

T: 100.2 C

po zmianie na krótszą wartość może zostać:

T: 9.2 C

Prosty wariant nadpisania:

```
lcd.setCursor(0, 0);  
lcd.print("T: ");  
lcd.print(temperature, 1);  
lcd.print(" C  ");
```

Dodatkowe spacje na końcu usuwają pozostałość poprzedniego tekstu.

Uwaga sprzętowa: część adapterów LCD I2C ma pull-upy magistrali I2C podciągnięte do napięcia zasilania modułu. ESP8266 pracuje na logice 3.3V. Jeżeli używany LCD jest zasilany z 5V i ma pull-upy do 5V, bezpieczniejszy wariant to moduł działający poprawnie przy 3.3V albo konwerter poziomów logicznych. Na zajęciach korzystaj z wariantu sprawdzonego dla stanowiska.

V. BMP180 + LCD na jednej magistrali

BMP180 był już używany wcześniej jako sensor temperatury i ciśnienia. W LAB 06 ważna jest integracja BMP180 z nowym elementem: LCD.

Schemat odpowiedzialności:

- **BMP180** daje dane,
- **LCD** pokazuje lokalny stan,
- **Serial** wysyła rekord do Bridge'a,
- **Bridge** wystawia `/reading`,
- **klient** mierzy odpowiedź HTTP.

Przykładowy układ danych w firmware:

```
float temperature = bmp.readTemperature();  
float pressureHpa = bmp.readPressure() / 100.0f;
```

Przykładowy układ wyświetlenia:

```
lcd.setCursor(0, 0);  
lcd.print("T: ");  
lcd.print(temperature, 1);  
lcd.print(" C ");  
  
lcd.setCursor(0, 1);  
lcd.print("P: ");  
lcd.print(pressureHpa, 0);  
lcd.print(" hPa ");
```

Przykładowy układ rekordu po Serial:

```
JsonDocument doc;  
  
doc["temperature"] = temperature;  
doc["pressureHpa"] = pressureHpa;  
  
serializeJson(doc, Serial);  
Serial.println();
```

Każdy rekord po `Serial` powinien kończyć się nową linią. Bridge czyta dane liniami. Brak `Serial.println()` po JSON-ie może dać po stronie komputera niepełny rekord albo timeout.

VI. `/reading` — co mierzysz naprawdę

Endpoint `/reading` jest wystawiany przez Bridge, nie przez samo ESP8266.

To oznacza, że pomiar `latencyMs` obejmuje odpowiedź lokalnego HTTP po stronie komputera. W praktyce sprawdzasz zachowanie toru:

klient pomiarowy -> Bridge HTTP -> ostatni znany odczyt z Serial

Jeżeli Bridge ma świeże dane z ESP, `/reading` powinien zwrócić JSON z temperaturą i ciśnieniem.

Przykładowa odpowiedź:

```
{  
  "temperature": 23.41,  
  "pressureHpa": 1011.92,  
  "unitTemperature": "C",  
  "unitPressure": "hPa",  
  "lastUpdateUtc": "2026-04-08T10:15:30Z",  
  "lastError": null  
}
```

Test podstawowy:

```
curl http://127.10.10.10:5080/reading
```

Jeżeli `/reading` nie odpowiada albo zwraca błąd, sprawdzaj warstwy w tej kolejności:

1. czy ESP wysyła JSON przez Serial,
2. czy port COM jest poprawny,
3. czy Bridge wystartował,
4. czy Bridge nie jest zablokowany przez Serial Monitor,
5. czy adres i port HTTP są poprawne,
6. czy klient odpytuje właściwy endpoint.

VII. Pomiar **latencyMs** w kliencie

latencyMs to czas od rozpoczęcia requestu do otrzymania odpowiedzi albo błędu. W tym labie mierzysz go po stronie klienta, bo to klient korzysta z endpointu HTTP.

Szkic idei w C#:

```
var watch = Stopwatch.StartNew();

try
{
    using var response = await http.GetAsync(url);
    watch.Stop();

    var statusCode = (int)response.StatusCode;
    var body = await response.Content.ReadAsStringAsync();

    // zapisz: statusCode, watch.ElapsedMilliseconds, success/error
}
catch (Exception ex)
{
    watch.Stop();

    // zapisz: błąd, watch.ElapsedMilliseconds, success=false
}
```

Do pomiaru używaj **Stopwatch**, nie **DateTime.Now** odejmowanego ręcznie. **Stopwatch** jest przeznaczony do mierzenia czasu wykonania fragmentu programu.

Wynik pojedynczego requestu nie wystarcza. Pojedynczy pomiar może trafić na przypadkowe opóźnienie, chwilową pracę systemu albo jednorazowy błąd. Dlatego w BASIC wykonujesz serię **20–30 requestów**.

Wynik, który warto zapisać dla każdej próby:

timestamp,endpoint,attempt,statusCode,latencyMs,retryCount,success,error

Przykładowy rekord:

2026-05-05T10:15:31.120Z,/reading,1,200,18,0,true,

Przykładowy rekord z błędem:

2026-05-05T10:15:35.840Z,/reading,1,503,31,1,false,ServiceUnavailable

VIII. Retry i backoff

Retry to ponowienie próby po błędzie. **Backoff** to odstęp przed ponowieniem.

W BASIC wystarczy prosty wariant:

- request do `/reading`,
- jeżeli wystąpi timeout, wyjątek transportowy albo kod HTTP spoza sukcesu — wykonaj jedno retry,
- zapisz `retryCount`,
- zapisz końcowy wynik próby.

Szkic idei:

```
int retryCount = 0;
bool success = false;

for (int attempt = 1; attempt <= 2; attempt++)
{
    var result = await TryReadAsync();
    if (result.Success)
    {
        success = true;
        break;
    }
    if (attempt == 1)
    {
        retryCount++;
        await Task.Delay(100);
    }
}
```

W logu rozróżniaj:

- sukces za pierwszym razem,
- sukces po retry,
- błąd mimo retry.

To są trzy różne sytuacje. Wszystkie mogą kończyć się podobnym widokiem „program działa”, ale nie oznaczają tej samej jakości toru.

W PERFORMANCE porównujesz dwa odstępy, np.:

- **100 ms,**
- **500 ms.**

Przy porównaniu nie patrz tylko na to, czy „przeszło”. Sprawdź:

- ile było sukcesów,
- ile było błędów,
- ile razy retry było potrzebne,
- jak zmienił się typowy czas odpowiedzi,
- czy najgorsze przypadki są akceptowalne.

IX. CSV/log — dane surowe są artefaktem

CSV albo log jest głównym śladem eksperymentu.

Minimalne pola:

Pole	Znaczenie
timestamp	kiedy wykonano próbę
endpoint	jaki endpoint był testowany
attempt	numer próby w serii
statusCode	kod HTTP, jeżeli odpowiedź dotarła
latencyMs	czas odpowiedzi w milisekundach
retryCount	liczba ponowień
success	końcowy wynik próby
error	krótki opis błędu

Przykładowy nagłówek CSV:

timestamp,endpoint,attempt,statusCode,latencyMs,retryCount,success,error

Przy zapisie CSV uważaj na przecinki w polu `error`. Najprostszy wariant: zapisuj krótki kod błędu bez przecinków, np.:

Timeout

Http503

JsonError

ConnectionRefused

Trzy wnioski powinny wynikać z danych. Przykładowe kierunki analizy:

- typowy czas odpowiedzi,
- pojedyncze wartości odstające,
- liczba błędów,
- ile razy retry było potrzebne,
- różnica między pomiarem spokojnym a burstem,
- różnica między backoff 100 ms i 500 ms.

X. Fallback: PowerShell na Windows 10

Wariant uproszczony powinien zapisać te same typy informacji: endpoint, czas, status, sukces albo błąd.

PowerShell ma dostęp do `System.Diagnostics.Stopwatch`, który pozwala zmierzyć czas wykonania pojedynczego żądania.

Szkic jednego pomiaru:

```
$sw = [System.Diagnostics.Stopwatch]::StartNew()

try {
    $response = Invoke-WebRequest -Uri $url -UseBasicParsing -TimeoutSec 3
    $sw.Stop()
}
catch {
    $sw.Stop()
}
```

Szkic dopisania rekordu do pliku:

```
"$timestamp,/reading,$i,$status,$latency,$retryCount,$success,$errorText" |
Add-Content -Path "measurements.csv"
```

Wariant PowerShell powinien zachować ten sam sens rekordu co wariant .NET:

```
timestamp,endpoint,attempt,statusCode,latencyMs,retryCount,success,error
```

Retry w PowerShell polega na tym samym schemacie: błąd pierwszej próby, krótki odstęp, druga próba, zapis końcowego wyniku. Nie mieszaj w jednym pliku pomiarów z różnymi formatami rekordu.

XI. Burst — krótkie obciążenie w PERFORMANCE

Burst to seria szybkich requestów w krótkim czasie. W tym labie przykładowy wariant to około:

20 req/s przez 10 s

To nie jest pełny benchmark systemu. To krótka próba zachowania endpointu pod większą liczbą wywołań.

Przy burst zapisz te same pola co w BASIC. Dzięki temu możesz porównać dane:

- BASIC spokojny pomiar,
- PERFORMANCE burst,
- backoff 100 ms,
- backoff 500 ms.

Porównanie ma sens tylko wtedy, gdy format danych jest ten sam. Jeżeli za każdym razem zapisujesz inne pola, analiza szybko staje się ręczna i nieczytelna.

XII. Wokwi + Wi-Fi jako wariant symulacyjny

W Wokwi najczęściej używa się **ESP32** do scenariuszy Wi-Fi. Dokumentacja Wokwi dla ESP32 Wi-Fi pokazuje połączenie przez sieć **Wokwi-GUEST**; własne access pointy są funkcją planów Hobby+/Pro, a użytkownicy free mogą używać domyślnej sieci **Wokwi-GUEST**. (docs.wokwi.com)

Minimalny fragment połączenia z siecią Wokwi:

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);

  WiFi.begin("Wokwi-GUEST", "");

  while (WiFi.status() != WL_CONNECTED) {
    delay(250);
    Serial.print(".");
  }

  Serial.println();
  Serial.println(WiFi.localIP());
}
```

Wariant z własnym SSID i hasłem wymaga dodania własnego access pointa w projekcie Wokwi. Dokumentacja oznacza **wokwi-wifi-ap** jako funkcję płatną dla planów Hobby+/Pro. (docs.wokwi.com)

Jeżeli w symulacji uruchamiasz serwer HTTP na ESP32 i chcesz otworzyć go z przeglądarki lub narzędzia HTTP poza symulatorem, sprawdź konfigurację **Wokwi IoT Gateway**. Wokwi pricing opisuje shared public IoT gateway w planie free oraz private IoT gateway w planach płatnych; prywatny gateway pozwala hostować własny HTTP server na symulowanym ESP32 i łączyć go z lokalnymi lub internetowymi urządzeniami. (wokwi.com)

Przy wariacie symulacyjnym zachowaj ten sam układ obserwacji:

- endpoint HTTP,
- seria requestów,
- `latencyMs`,
- `statusCode`,
- `success/error`,
- CSV/log.

XIII. UDP telemetry

UDP nie działa jak HTTP.

W HTTP klient pyta, a serwer odpowiada:

request -> response

W UDP nadawca wysyła datagram. Nie ma gwarancji dostarczenia, kolejności ani potwierdzenia.

Do eksperymentu UDP potrzebujesz numeru sekwencyjnego:

```
{  
  "seq": 42,  
  "temperature": 23.4,  
  "pressureHpa": 1012.1  
}
```

Po stronie odbiornika zapisujesz:

```
receivedAt, seq, temperature, pressureHpa
```

Straty sprawdzasz po przerwach w **seq**:

```
40, 41, 42, 44, 45
```

W tym przykładzie brakuje pakietu 43.

Jitter oceniasz przez odstępy między odebranymi pakietami. Jeżeli pakiety miały przychodzić co 100 ms, a odbiornik widzi odstępy:

98 ms, 101 ms, 147 ms, 82 ms, 104 ms

to odstępy nie są równe. To jest punkt wyjścia do rozmowy o zmienności opóźnień.

XIV. Red/Blue lens: retry i przeciążenie

Retry pomaga przy błędach chwilowych. Ten sam mechanizm może też zwiększyć presję na system.

Przykład:

- endpoint zaczyna odpowiadać wolniej,
- klient dostaje timeout,
- klient ponawia request,
- kilka klientów robi to samo,
- Bridge albo urządzenie dostają jeszcze więcej pracy.

Pytanie na koniec labu:

Czy retry w Twoim pomiarze poprawiło odporność, czy tylko ukryło problem pierwszej próby?

Drugie pytanie:

Co stanie się, jeśli każdy klient będzie ponawiał request natychmiast po błędzie?

To jest red/blue lens dla LAB 06. Mechanizm techniczny może poprawiać doświadczenie klienta, ale może też pogarszać sytuację przy przeciążeniu.

XV. Checklista „nie działa”

Jeżeli coś nie działa, przejdź po tej kolejności:

1. Czy **BMP180** ma poprawne zasilanie **3.3V**, **GND**, **SDA**, **SCL**?
2. Czy **LCD** ma poprawne zasilanie, **GND**, **SDA**, **SCL**?
3. Czy masa jest wspólna dla Wemos D1, BMP180 i LCD?
4. Czy SDA i SCL nie są zamienione?
5. Czy LCD ma właściwy adres I2C: **0x27** albo **0x3F**?
6. Czy kontrast LCD jest ustawiony potencjometrem?
7. Czy **Wire.begin(4, 5)** odpowiada użytym pinom?
8. Czy **Serial Monitor** pokazuje poprawny JSON albo czytelny błąd?
9. Czy Bridge używa właściwego portu COM i prędkości?
10. Czy Serial Monitor nie blokuje portu COM, gdy Bridge chce go czytać?
11. Czy **/reading** odpowiada przez **curl** albo **.http**?
12. Czy klient mierzy właściwy URL?
13. Czy CSV/log zapisuje również błędy, a nie tylko sukcesy?
14. Czy retry jest widoczne w **retryCount**?
15. Czy wnioski wynikają z danych, które masz w pliku?

Najpierw sprawdź warstwę fizyczną i lokalny pomiar. Potem **Serial**. Potem Bridge. Potem HTTP. Dopiero na końcu klienta i CSV.

Źródła i linki

Jeśli utknąłeś, nie "zgaduj". Inżynier zagłąda do specyfikacji:

Poziom Zadania	Rozwiązany Problem	Link ratunkowy (Source of Truth)
● BASIC	Podstawy pracy z płytą prototypową: rzędy, szyny zasilania, masa, połączenia bez lutowania	SparkFun Learn -> How to Use a Breadboard . Dobry punkt startowy do zrozumienia, jak fizycznie połączone są otwory na breadboardzie i jak używać szyn zasilania.
● BASIC	Breadboard od strony początkującego użytkownika: typowe układy, wewnętrzne połączenia, podstawowe zasady użycia	Adafruit Learn -> Breadboards for Beginners . Przydatne, gdy chcesz szybko sprawdzić, jak działa płyta stykowa i dlaczego przewód włożony „obok” nie zawsze oznacza połączenie.
● BASIC	Magistrala I2C na ESP8266 i jawne wskazanie linii SDA / SCL	Dokumentacja ESP8266 Arduino Core -> I2C / Wire library . Source of truth dla <code>Wire.begin(sda, scl)</code> oraz domyślnych pinów <code>GPIO4</code> / <code>GPIO5</code> na ESP8266.

Poziom Zadania	Rozwiązany Problem	Link ratunkowy (Source of Truth)
● BASIC	Ogólne działanie biblioteki Wire i komunikacji I2C	Arduino Documentation -> Wire . Przydatne, gdy chcesz sprawdzić podstawowy model pracy magistrali I2C i funkcje używane do komunikacji z urządzeniami.
● BASIC	Odczyt temperatury i ciśnienia z BMP180 / BMP085	GitHub -> Adafruit BMP085 Library . Biblioteka dla czujników BMP085/BMP180, używana do odczytu temperatury i ciśnienia po I2C.
● BASIC	Przykład użycia biblioteki BMP085/BMP180	GitHub -> BMP085test.ino . Przydatne do sprawdzenia nazw metod i ogólnego wzorca inicjalizacji czujnika.
● BASIC	Obsługa wyświetlacza LCD 2x16 przez adapter I2C	Arduino Libraries -> LiquidCrystal I2C . Dokumentacja biblioteki do sterowania wyświetlaczami LCD przez I2C funkcjami podobnymi do klasycznego LiquidCrystal .
● BASIC	Alternatywna biblioteka dla LCD I2C / PCF8574	Arduino Libraries -> LCD-I2C . Przydatne, gdy używany moduł LCD korzysta z adaptera I2C opartego o PCF8574 i wymaga innej biblioteki niż standardowo zakładana.
● BASIC	Serializacja JSON po stronie ESP8266	Dokumentacja biblioteki -> ArduinoJson 7 Documentation . Punkt odniesienia dla budowania rekordu JSON wysyłanego przez Serial .

Poziom Zadania	Rozwiązany Problem	Link ratunkowy (Source of Truth)
● BASIC	JsonDocument i model pracy w ArduinoJson v7	ArduinoJson API -> JsonDocument . Przydatne przy budowaniu obiektu JSON bez ręcznego składania tekstu.
● BASIC	Obsługa portu szeregowego po stronie Bridge'a .NET	Microsoft Learn -> SerialPort Class . Source of truth dla pracy z portem szeregowym: port COM, baud rate, odczyt danych i operacje I/O.
● BASIC	Odczyt linii z portu szeregowego po stronie .NET	Microsoft Learn -> SerialPort.ReadLine Method . Przydatne, gdy Bridge czyta rekordy JSON jako kompletne linie zakończone znakiem nowej linii.
● BASIC	Lokalny Bridge HTTP w ASP.NET Core Minimal API	Microsoft Learn -> Tutorial: Create a Minimal API with ASP.NET Core . Fundament dla projektu Bridge'a wystawiającego endpointy typu <code>/reading</code> .
● BASIC	Testowanie endpointu <code>/reading</code> z terminala	curl documentation -> curl tutorial . Przydatne do ręcznego sprawdzenia endpointu przed uruchomieniem klienta pomiarowego.
● BASIC	Znaczenie kodów HTTP, w tym 200 i 503	MDN Web Docs -> HTTP response status codes . Punkt odniesienia dla interpretacji odpowiedzi HTTP w logu pomiarowym.
● BASIC	Co oznacza 503 Service Unavailable	MDN Web Docs -> 503 Service Unavailable . Przydatne, gdy Bridge działa, ale nie ma poprawnego albo świeżego odczytu z toru pomiarowego.

Poziom Zadania	Rozwiązany Problem	Link ratunkowy (Source of Truth)
● BASIC / ● PERFORMANCE	Wysyłanie żądań HTTP w kliencie .NET	Microsoft Learn -> Make HTTP requests with HttpClient . Source of truth dla <code>HttpClient</code> , odpowiedzi HTTP i podstawowej obsługi żądań.
● BASIC / ● PERFORMANCE	Pomiar czasu wykonania fragmentu programu w .NET	Microsoft Learn -> Stopwatch Class . Przydatne przy pomiarze <code>latencyMs</code> po stronie klienta.
● BASIC / ● PERFORMANCE	<code>ElapsedMilliseconds</code> w <code>Stopwatch</code>	Microsoft Learn -> Stopwatch.ElapsedMilliseconds . Konkretnie odniesienie dla zapisu czasu odpowiedzi w milisekundach.
● BASIC / ● PERFORMANCE	Fallback pomiarowy w PowerShell: żądania HTTP	Microsoft Learn -> Invoke-WebRequest . Przydatne, jeśli wykonujesz prosty wariant pomiaru endpointu bez klienta C#.
● BASIC / ● PERFORMANCE	Zapis danych do CSV w PowerShell	Microsoft Learn -> Export-Csv . Przydatne, gdy chcesz zapisać wyniki pomiarów do pliku CSV z poziomu PowerShell.
● PERFORMANCE	Krótki test burst i interpretacja statusów HTTP pod większą liczbą wywołań	MDN Web Docs -> HTTP response status codes . Ten sam słownik statusów przydaje się przy porównaniu spokojnego pomiaru i krótkiego obciążenia.
● PERFORMANCE	Retry, opóźnienie i zachowanie klienta HTTP	Microsoft Learn -> Guidelines for using HttpClient . Przydatne przy myśleniu o wielokrotnym odpytywaniu endpointu i zachowaniu klienta HTTP.

Poziom Zadania	Rozwiązany Problem	Link ratunkowy (Source of Truth)
● PERFORMANCE / ● DELIGHTERS	Symulacja Wi-Fi w Wokwi na ESP32	Wokwi Docs -> ESP32 WiFi Networking . Dokumentacja połączenia przez Wokwi-GUEST, publiczny/prywatny gateway oraz podstawowy model pracy z Wi-Fi w symulatorze.
● PERFORMANCE / ● DELIGHTERS	Własny access point w Wokwi i ograniczenia planów	Wokwi Docs -> wokwi-wifi-ap Reference . Przydatne przy sprawdzaniu, kiedy działa domyślne Wokwi-GUEST, a kiedy potrzebny jest własny access point.
● PERFORMANCE / ● DELIGHTERS	Aktualne ograniczenia planów Wokwi i IoT Gateway	Wokwi -> Pricing . Punkt do sprawdzenia, jakie funkcje gateway / Wi-Fi są dostępne w planie free, a jakie w planach płatnych.
● DELIGHTERS	UDP na ESP8266	Dokumentacja ESP8266 Arduino Core -> UDP Class . Source of truth dla klasy WiFiUDP i podstaw pracy z UDP na ESP8266.
● DELIGHTERS	Przykłady UDP na ESP8266	Dokumentacja ESP8266 Arduino Core -> UDP examples . Przydatne przy budowie prostego nadawcy lub odbiornika pakietów UDP.
● DELIGHTERS	Analiza ruchu sieciowego i filtry w Wiresharku	Wireshark User's Guide -> Wireshark User's Guide . Punkt startowy do oglądania ruchu HTTP / UDP i pracy z filtrami.

Na kolejny lab (LAB 07 - Integracja końcowa: Evidence Room Surveillance Node)

Cel: Przejście od eksperymentu pomiarowego do integracji całego toru systemowego. W następnym laboratorium zamykamy wprowadzanie nowych narzędzi i składamy projekt końcowy: wirtualny węzeł **ESP32** będzie dostarczał dane i zdarzenia, a aplikacja serwerowa **C# Edge Gateway** przejmie rolę warstwy decyzyjnej. Będziesz łączyć akwizycję danych, kontrakt HTTP, logikę wykrywania anomalii, autoryzację żądań oraz ślad audytowy w jeden spójny scenariusz.

1. **Co musisz dokończyć w domu?** Zadania **BASIC** z LAB 06 są obowiązkowe — tor **BMP180** → **ESP8266** → **Serial** → **Bridge** → **HTTP**, dołożony **LCD 2x16 I2C**, uporządkowane połączenia na płytce prototypowej oraz endpoint **/reading** muszą działać poprawnie. Plik **CSV/log** z pomiarami **latencyMs**, **retryCount**, statusem odpowiedzi i błędami powinien być zachowany razem z kodem. Trzy krótkie wnioski techniczne z LAB 06 traktuj jako zamknięcie eksperymentu: na kolejnym spotkaniu nie będziemy już wracać do pytania, czy umiesz wykonać pomiar endpointu i odróżnić wynik od wrażenia. Zadania **PERFORMANCE** z LAB 06 — burst oraz porównanie backoff **100 ms vs 500 ms** — są rekomendowane, bo pomagają lepiej zrozumieć zachowanie systemu pod presją. Ścieżka **DELIGHTER** z UDP i analizą strat/jitter pozostaje opcjonalna.
2. **Zapowiedź: Co będziemy budować?** W **LAB 07** zaczniecie budować projekt zaliczeniowy „**Nadzór Archiwum Dowodowego**” (**Evidence Room Surveillance Node**) — system wspierający nadzór nad kontrolowaną strefą, w której każde podejrzanе zdarzenie powinno zostać wykryte, ocenione i zapisane w śladzie audytowym. Celem projektu będzie przygotowanie działającego scenariusza, który łączy węzeł pomiarowy z aplikacją serwerową i pozwala przejść od pojedynczych odczytów do uporządkowanej decyzji systemowej. Projekt będzie można dokończyć w domu.
3. **Zastanów się nad integracją toru danych:**
 1. **Gdzie kończy się pomiar, a zaczyna decyzja?** Czy węzeł ma tylko wysłać dane, czy powinien samodzielnie interpretować ich znaczenie? Która warstwa powinna uznać, że spadek ciśnienia jest anomalią?
 2. **Kontrakt danych:** Jak powinien wyglądać payload, żeby aplikacja serwerowa mogła odróżnić zwykły odczyt od stanu wymagającego reakcji? Jakie pola są potrzebne, a jakie tylko utrudniają analizę?
 3. **Stan systemu:** Czy pojedynczy odczyt wystarczy do wykrycia anomalii, czy potrzebujesz porównania z poprzednim stanem? Co oznacza „gwałtowny spadek” w danych, a nie w intuicji?

4. Zastanów się nad odpornością i śladem audytowym:

1. **Audit trail:** Co powinno zostać zapisane, gdy system wykryje naruszenie strefy? Sam komunikat **ALERT** nie wystarczy, jeśli później trzeba odtworzyć przebieg zdarzenia.
2. **Autoryzacja żądań:** Co powinno się stać, gdy Edge Gateway otrzyma request bez poprawnego tokenu albo z nieprawidłowym identyfikatorem urządzenia? Czy taki request powinien trafić do logiki decyzyjnej?
3. **Red/blue lens:** Jeżeli zwykły HTTP pokazuje payload i nagłówki w jawnej postaci, to co naprawdę chroni prosty token? Które ryzyka ogranicza, a których nie rozwiązuje?